Agentic AI Based Animal Rescue Management System

Dissertation submitted to

Shri Ramdeobaba College of Engineering & Management, Nagpur in partial fulfilment of requirement for the award of degree of

Bachelor of Technology (B.Tech)

In

COMPUTER SCIENCE AND ENGINEERING

(Artificial Intelligence & Machine Learning)

By

Adwait Dubey (16)

Himanshu Pandey (29)

Mithanshu Hedau (37)

Raj Khatri (45)

Sumedh Khangan (57)

of

VIII Semester

Guided By

Dr Suresh Balpande .

RCOEM

Shri Ramdeobaba College of Engineering and Management, Nagpur

Department of Computer Science and Engineering (Artificial Intelligence & Machine Learning)

Shri Ramdeobaba College of Engineering & Management, Nagpur 440 013

(An Autonomous Institute affiliated to Rashtrasant Tukdoji Maharaj Nagpur University Nagpur)

April 2026

SHRI RAMDEOBABA COLLEGE OF ENGINEERING MANAGEMENT, NAGPUR

(An Autonomous Institute affiliated to Rashtrasant Tukdoji Maharaj Nagpur University Nagpur)

Department of Artificial Intelligence and Cyber Security (AICS)

CERTIFICATE

This is to certify that the Thesis on "Agentic AI Based Animal Rescue Management System" is a bonafide work of Adwait Dubey, Himanshu Pandey, Mithanshu Hedau, Sumedh Khangan, and Raj Khatri, submitted to the Rashtrasant Tukadoji Maharaj Nagpur University in partial fulfilment of the award of a Degree of Bachelor of Technology (B.Tech) in Computer Science and Engineering (AIML).

It has been carried out at the Department of Artificial Intelligence and Cyber Security (AICS), Shri Ramdeobaba College of Engineering and Management during the academic year 2025–2026.

Date: 09/05/2026

Place: Nagpur

Dr.Suresh Balpande
Project Guide

Dr. Rashmi Welekar
HOD

Department of Artificial Intelligence and
Cyber Security

Dr. M. B. Chandak
Principal

DECLARATION

We hereby declare that the thesis titled **"Agentic AI Based Animal Rescue Management System"** submitted herein, has been carried out in the Department of **Artificial Intelligence and Cyber Security (AICS)** of Shri Ramdeobaba College of Engineering and Management.

The work is original and has not been submitted earlier as a whole or in part for the award of any degree/diploma at this or any other institution/university.

Date: 09/05/26

Place: Nagpur

Name, roll no and sign of all students

Adwait Dubey (16)

Himanshu Pandey (29)

Mithanshu Hedau (37)

Raj Khatri (45)

Sumedh Khangan (57)

APPROVAL SHEET

This report entitled **"Agentic AI Based Animal Rescue Management System"** by **Adwait Dubey, Himanshu Pandey, Mithanshu Hedau, Sumedh Khangan, and Raj Khatri** is approved for the degree of **Bachelor of Technology (B.Tech)**.

Dr.Suresh Balpande
Project Guide

External Examiner

Dr. R. Welekar
H.O.D, AICS

Date: 09/05/2026
Place: Nagpur

4

ACKNOWLEDGEMENT

We would like to express our deep and sincere gratitude to our guide Dr. Suresh Balpande for enabling us to work on this project and for offering valuable advice and continuous support throughout the development of this work. Working under his guidance was a great privilege and honour. We are truly thankful for the opportunity to carry out this project under his mentorship.

We also express our sincere gratitude to Dr. Rashmi Welekar, Head of the Department of Artificial Intelligence and Cyber Security (AICS), Shri Ramdeobaba College of Engineering and Management, for her guidance and encouragement throughout the project.

We would also like to extend our heartfelt gratitude to Dr. M. B. Chandak, Principal of Shri Ramdeobaba College of Engineering and Management, for providing us with the opportunity, facilities, and academic environment that greatly supported us in the successful completion of this project.

Talent wins games, but teamwork and intelligence win championships. We would like to take this opportunity to express our heartfelt gratitude to everyone who directly or indirectly helped us in successfully completing this project.

ABSTRACT

Animal rescue operations require rapid response, accurate coordination, and continuous monitoring to ensure timely medical assistance and protection for injured, abandoned, or endangered animals. However, traditional rescue management systems—often dependent on manual reporting, phone-based communication, and fragmented record keeping—struggle to operate efficiently under real-world conditions involving emergency situations, delayed response times, lack of resource coordination, and limited accessibility to veterinary support. To address these challenges, this project introduces an **Agentic AI Based Animal Rescue Management System** designed to provide intelligent, real-time, and automated rescue coordination.

The proposed system integrates autonomous AI agents, predictive analytics, and humanin-the-loop (HITL) validation to support efficient and reliable decision-making during rescue operations. A dedicated Monitoring and Reporting Unit continuously gathers realtime information such as animal distress reports, location data, rescue team availability, medical requirements, and shelter capacity. This information is processed by intelligent reasoning agents that dynamically prioritize rescue cases based on urgency, injury severity, environmental conditions, and resource availability, enabling context-aware rescue planning instead of static manual coordination.

To further improve operational efficiency, the system incorporates AI-driven image analysis and predictive models to identify animal conditions and estimate the urgency of medical intervention. Automated communication mechanisms are integrated to notify nearby rescue volunteers, veterinary staff, and shelters through email and alert-based coordination, thereby minimizing manual workload and response delays. For critical rescue situations involving severe injuries or legal concerns, a HITL governance layer ensures supervisory verification and approval before finalizing rescue or medical decisions.

Simulation and testing results demonstrate significant improvements in rescue efficiency, including reduced response time, improved coordination accuracy, faster emergency reporting, and enhanced transparency in rescue operations. Overall, the proposed Agentic AI framework establishes a scalable, intelligent, and reliable foundation for modern animal rescue and welfare management, offering a substantial advancement over conventional manual rescue systems.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1

INTRODUCTION

Stray animal populations across India represent a significant and growing public health, welfare, and ecological concern. Millions of cats, dogs, and cattle roam urban and semi-urban environments without access to veterinary care, vaccination, or nutritional support. Many of these animals carry contagious diseases that are transmissible to humans and other animals, yet the absence of accessible, real-time identification and reporting infrastructure means that the vast majority go untreated. Animal rescue organisations, municipal veterinary departments, and individual volunteers lack the tools to rapidly assess an animal's species and health condition in the field, leading to delayed intervention, missed diagnoses, and preventable suffering.

Traditional approaches to stray animal welfare rely on manual reporting through phone calls or in-person visits to NGO offices, visual inspection by volunteers with no veterinary training, and paper-based case logging that is slow, error-prone, and difficult to coordinate across multiple stakeholders. These methods are reactive by nature — a rescue only begins after a citizen notices a suffering animal and manages to reach the right authority. By that point, a disease may have progressed significantly or spread to neighbouring animals.

To address these limitations, this project proposes **Agentic AI Based Animal Rescue Management System** — an AI-powered stray animal rescue and disease detection platform that enables any citizen to photograph a stray animal and receive an immediate, AI-driven assessment of the animal's species and probable disease condition. The system integrates a ResNet-18 based animal classifier trained in PyTorch, three specialised EfficientNetB0-based disease detection models for cats, cows, and dogs trained in TensorFlow/Keras, a AI conversational assistant powered by Google-gemeni for crosschecks, and a real-time NGO location and WhatsApp-based reporting pipeline. Together, these components transform a photograph taken in the field into a structured rescue report that can be dispatched to a nearby animal welfare organisation in seconds.

The platform is delivered as a responsive web application built with Flask, HTML, CSS, and Vanilla JavaScript, requiring no mobile app installation and functioning on any smartphone browser — a deliberate design choice to maximise accessibility for volunteers in lowresource environments.

1.1 Background

The stray animal crisis in India is characterised by scale, complexity, and a persistent gap between the number of animals requiring assistance and the capacity of existing welfare infrastructure to respond. According to the Animal Welfare Board of India, the country has one of the largest stray dog populations in the world, with tens of millions of unvaccinated animals living in close proximity to human settlements. Similar challenges exist for stray cattle displaced from agricultural areas and for feral cat colonies in urban environments.

Animal diseases prevalent among stray populations include:

- Dermatological conditions such as Ringworm, Scabies, Demodicosis, and Fungal Infections that spread rapidly through direct contact

- Systemic viral diseases including Feline Leukemia, Feline Panleukopenia, and Foot and Mouth Disease that carry high mortality rates if untreated

- Parasitic and bacterial infections including Ear Mites, Worm Infections, Eye Infections, and Urinary Tract Infections that significantly reduce quality of life

- Hypersensitivity and allergic conditions that are difficult to distinguish visually from infectious skin disease without veterinary examination

Historically, disease identification in stray animals required physical examination by a trained veterinarian — a resource that is scarce, expensive, and geographically concentrated in urban centres. Volunteers and citizens who encounter suffering animals have no reliable tool for even preliminary triage, making it difficult to communicate the urgency or nature of a case to rescue organisations.

The emergence of deep learning-based image classification, combined with the availability of pre-trained convolutional neural network architectures such as ResNet and EfficientNet, has created an opportunity to democratise preliminary animal health assessment. Transfer learning from large-scale image datasets such as ImageNet enables highly accurate disease classification to be achieved with relatively modest domain-specific training data — making it feasible to build specialised models for each animal species rather than relying on a single generalised classifier.

cameras, combined with the near-universal adoption of WhatsApp for community communication in India, creates an ideal distribution channel for an AI-assisted rescue platform that generates structured, shareable reports from a single photograph.

1.2 Motivation

Animal welfare in India is fundamentally constrained by an information gap — volunteers and citizens who encounter stray animals in distress often lack the knowledge to identify what disease an animal may be suffering from, which organisation to contact, and how to communicate the case effectively. This gap results in delayed rescue, inappropriate first aid, and missed opportunities for early intervention that could prevent disease progression and cross-species transmission.

Several compounding challenges motivate the development of an AI-assisted solution:

Existing reporting mechanisms are fragmented and slow. Most NGOs rely on WhatsApp messages or phone calls to receive rescue requests, with no standardised format for capturing animal species, condition, or location. This makes case prioritisation difficult and follow-up unreliable.

Volunteer capacity is insufficient relative to need. The number of trained animal welfare workers is a fraction of what is required to address the scale of stray populations in any major Indian city. Empowering untrained citizens to perform reliable preliminary triage using AI could multiply the effective capacity of the rescue ecosystem without requiring additional trained staff.

Disease transmission represents a genuine public health risk. Rabies, Ringworm, and certain bacterial infections are zoonotic — they can spread from stray animals to humans. Early identification and isolation of diseased animals requires rapid detection tools that can operate at the point of first encounter rather than waiting for a veterinarian to visit.

NGO coordination is geographically inefficient. Rescue organisations often operate within specific zones, yet reports are routed through central call centres that may not be aware of which field unit is closest to the incident. Real-time location-based NGO mapping can significantly reduce response times.

The absence of a unified digital record of reported animals makes it impossible to track which cases have been resolved, which animals have been vaccinated, and where disease clusters are forming. A platform that logs reports and maintains a visible dashboard of case status addresses this systemic gap.

These challenges, when examined collectively, reveal critical gaps in the current stray animal welfare infrastructure that directly motivated the development of StrayAid:

1. **No accessible AI triage tool** — Citizens have no way to assess a stray animal's condition beyond visual guesswork, and no standardised channel to communicate that assessment to a rescue organisation.

2. **Species-specific disease complexity** — Disease presentations differ significantly across cats, dogs, and cattle, requiring specialised models rather than a single generalised classifier.

3. **Low confidence in AI outputs** — A single model prediction without contextual confirmation creates risk of misclassification acting on. A conversational AI confirmation step reduces this risk before a report is dispatched.
4. **Lack of real-time NGO visibility** — Volunteers do not know which rescue organisation is closest to their location or how to contact them instantly.

5. **No structured reporting format** — Rescue reports communicated verbally or through informal messages lack the structured data (species, disease, GPS coordinates, AI confidence) that organisations need to prioritise and coordinate response.

6. **No unified case tracking** — There is no dashboard visibility into which animals have been reported, what condition they are in, and whether rescue has been completed.

To address these limitations, an intelligent, image-driven platform capable of species identification, disease screening, conversational AI confirmation, location-based NGO discovery, and structured WhatsApp reporting becomes essential. The StrayAid platform proposed in this study aims to deliver faster rescue initiation, reliable preliminary diagnosis, transparent case documentation, and proactive community coordination through a combination of deep learning, transfer learning, and large language model integration.

1.3 Objectives

- Develop a two-stage AI pipeline using a ResNet-18 animal classifier and three EfficientNetB0 disease detection models (cat, cow, dog) capable of identifying animal species and probable disease condition from a single uploaded photograph.

- Design a clean, responsive desktop web interface that enables citizens, volunteers, and NGO workers to upload images, receive AI analysis results, and dispatch structured rescue reports without requiring any technical knowledge or application installation.

- Integrate a Paw AI conversational assistant powered by the Claude large language model to enable users to confirm AI findings through natural language dialogue before a report is sent, reducing the risk of acting on a misclassification.

- Implement real-time GPS-based and address-based location detection using the Browser Geolocation API and OpenStreetMap Nominatim geocoding to identify the user's position and display nearby NGOs and animal welfare agencies on an interactive Leaflet.js map.
- Enable one-tap WhatsApp reporting that packages the animal species, detected disease, AI confidence scores, Paw AI assessment summary, GPS coordinates, and Google Maps link into a structured message dispatched directly to an NGO or rescue contact.

- Present disease screening results in plain, accessible natural language rather than raw confidence scores, so that non-expert users can understand and act on the AI output without veterinary training.

- Build a rescue dashboard that logs every reported animal with its species, disease status, location, and rescue progress, providing NGO workers and platform administrators with real-time visibility into active cases.

- Design the Flask backend with lazy-loaded model inference to minimise startup time, and structure the codebase to support future integration of database persistence, user authentication, and multi-user NGO coordination features.
- Ensure the platform is extensible for future enhancements including lost and found animal matching, community volunteer feeds, vet consultation booking, and IoT-based stray population mapping.

1.4 Technologies Used

The platform integrates a range of technologies spanning deep learning, web development, mapping, natural language processing, and automated communication. Below are the key technologies used:

I. PyTorch and torchvision

- Deep learning framework used to implement and run the ResNet-18 animal classification model.
- Handles image preprocessing through the transforms pipeline and performs GPU/CPUadaptive inference using torch.no_grad() for efficient prediction.

II. TensorFlow and Keras

- Used to build, train, and run the three EfficientNetB0-based disease detection models for cats, cows, and dogs.
- Keras provides the high-level API for model construction, data loading via image_dataset_from_directory, training callbacks (ModelCheckpoint, EarlyStopping), and saved model serialisation to .keras format.

III. EfficientNetB0 and ResNet-18 (Transfer Learning)

- Pre-trained convolutional neural network architectures initialised with ImageNet weights and fine-tuned on domain-specific animal disease datasets.
- Transfer learning enables high classification accuracy with relatively small training datasets by reusing feature extraction layers learned from 1.2 million images.

IV. Flask

- Lightweight Python web framework that serves the frontend HTML interface and exposes the /predict REST API endpoint.
- Implements lazy model loading to minimise server startup time, and will support future /report, /chat, and /auth endpoints as database and authentication features are integrated.

V. Google Gemini API

- The Google Gemini API is used as a cross-validation layer in the system to verify the disease predictions generated by the animal disease detection models. Since animal wound and injury datasets are limited and sensitive data is not widely available online, Gemini helps improve prediction reliability by analyzing the uploaded image along with the detected animal type, disease label, and confidence scores.

- Using its multimodal understanding capabilities, Gemini checks whether the predicted disease or wound matches the visible symptoms in the image before forwarding the verified result to the next stages of the system. The API is integrated through the Flask backend to ensure the Gemini API key remains secure and hidden from the frontend.

VI. Leaflet.js and OpenStreetMap

- Leaflet.js is an open-source JavaScript library used to render the interactive NGO location map.
- OpenStreetMap provides the map tile imagery (roads, terrain, city labels) entirely free of charge and without API key requirements.
- Nominatim, OpenStreetMap's geocoding service, converts manually entered address text into latitude and longitude coordinates for map centring.

VII. Browser Geolocation API

- Built into every modern browser, used to obtain the user's GPS coordinates when the "Get My Location" button is activated.
- Coordinates are used both for map centring and for inclusion in the WhatsApp rescue report as a Google Maps link.

VIII. WhatsApp URL Scheme

- The standard wa.me deep link format pre-populates a WhatsApp message with the complete structured rescue report including animal species, disease, AI confidence, Paw AI summary, GPS coordinates, and timestamp.
- Requires no WhatsApp API account or Business API subscription — the link opens directly in the WhatsApp application on any device.

IX. Pillow (PIL) and NumPy

- Pillow opens uploaded image files from raw bytes without writing to disk, and handles RGB conversion for consistency across PNG, JPEG, and WEBP inputs.
- NumPy performs array operations for Keras model preprocessing: normalising pixel values to 0–1 range, expanding batch dimensions, and extracting the highestprobability class using argmax.

X. HTML5, CSS3, and Vanilla JavaScript

- The full frontend interface is implemented without any JavaScript framework, using CSS Grid and Flexbox for the desktop dashboard layout, CSS custom properties for the design system, and the Fetch API with async/await for communication with the Flask backend.
- Google Fonts (Outfit and Syne) provide the typographic identity of the platform.

1.5 Use Cases

Use Case 1 — Volunteer Reporting a Sick Animal

A volunteer notices a stray animal showing visible symptoms of illness such as skin infection, wounds, or unusual behaviour. The volunteer opens the StrayAid Rescue screen and uploads an image of the animal. The system identifies the animal species and detects the most probable disease with associated confidence scores. Paw AI then initiates a conversational assessment by asking symptom-related follow-up questions. Based on the responses, the system recommends whether rescue or veterinary attention is required. Upon confirmation, a rescue report containing disease details, confidence score, timestamp, GPS coordinates, and map link is automatically dispatched to the nearest NGO through WhatsApp.

Use Case 2 — NGO Worker Monitoring Rescue Cases

An NGO worker accesses the StrayAid dashboard to monitor active rescue reports submitted by users. The dashboard displays reported animals along with detected diseases, locations, timestamps, and rescue statuses. The worker reviews individual cases, analyses the AI-generated assessments, and coordinates rescue actions with field teams. Cases that are resolved or identified as non-critical can be updated or marked as closed through the dashboard interface.

Use Case 3 — Researcher Evaluating System Performance

A researcher uses StrayAid to evaluate the performance of the animal classification and disease detection models. Multiple test images containing different animal species and disease conditions are uploaded through the Rescue interface. For each image, the system performs species identification, disease prediction, and conversational analysis using Paw AI. The researcher records prediction accuracy, confidence scores, and system responses for evaluation and documentation purposes.

Use Case 4 — Veterinary Officer Using Location-Based Assistance

A veterinary officer uses StrayAid to locate nearby rescue organisations during an emergency animal case. After enabling location services on the Rescue screen, the system displays nearby NGOs and veterinary agencies on an interactive map. The officer selects the nearest agency, reviews its contact information, and dispatches a pre-generated rescue report containing animal details, disease prediction, GPS coordinates, and timestamp directly through WhatsApp.

CHAPTER 2

LITERATURE REVIEW

The increasing number of stray animals, delayed rescue response, lack of disease awareness, and poor coordination between NGOs and citizens have created a major challenge in animal welfare systems. Existing solutions mainly focus on either reporting stray animals, disease detection, or rescue coordination individually. Very few systems combine Artificial Intelligence (AI), disease identification, rescue tracking, NGO connectivity, and community participation into a single platform.

Recent research in computer vision and deep learning has shown significant improvements in animal detection and veterinary diagnosis using Convolutional Neural Networks (CNNs), YOLO models, and image-based classification techniques [1], [2], [3]. Similarly, mobile applications and GIS-based systems have improved animal reporting and location tracking [4], [5], [6]. However, most available systems still suffer from limited datasets, lack of real-time emergency communication, absence of integrated NGO coordination, and no intelligent rescue workflow.

Our proposed system aims to bridge these gaps by developing an AI-powered Animal Rescue and Health Monitoring Platform that integrates:

- Animal classification and disease detection
- WhatsApp-based emergency alert system
- NGO and veterinary clinic connectivity
- Lost & Found animal tracking
- Community volunteer participation

This literature survey analyzes existing applications, research papers, and platforms related to animal rescue, disease detection, and stray monitoring systems.

2.1 Existing Applications & Research Analysis

Paper / System (Year)	**Key Work Done**	**Relevance to Project**	**Research Gap / Our Contribution**
Spot a Stray (Live)	Stray animal reporting platform (Lost & Found)	Covers reporting aspect of rescue	Supports only missing reports, no AI disease detection or NGO workflow

Stray Shedders (Live)	Rescue and stray reporting platform	Includes rescue coordination	Manual data entry, no AI prediction, no NGO tracking
YOLOv5 Stray Dog Detection (2025) – Elsevier [1]	AI-based stray dog detection using YOLOv5	Core object detection module	No health analysis, disease prediction, or rescue coordination
Injury & Lameness Detection (2023) – Elsevier [2]	CNN-based injury and lameness identification	Useful for health severity estimation	Works mainly for known breeds, less accurate for stray animals
InfoFaunaFVG (2023) – Springer [4]	GIS-based wildlife reporting system	Base architecture for reporting systems	No SOS alert, NGO workflow, or rescue tracking
Deep Learning in Veterinary Medicine (2021) – Frontiers [7], [3]	CNN and imaging-based disease diagnosis	Supports AI disease prediction model	Requires large and diverse datasets for practical deployment

Table 2.1— Literature Survey

2.2 Traditional Animal Rescue Systems

Earlier animal rescue systems mainly depended on **manual reporting methods** such as phone calls, local NGO records, social media posts, and offline veterinary consultations [4], [8]. In these systems, users had to personally contact rescue organizations or clinics to report injured or lost animals. Although these methods were simple and easy to use, they lacked technological support and proper coordination mechanisms.

Traditional systems did not provide advanced features like:

- Real-time tracking
- Centralized reporting
- Automated emergency communication
- AI-based disease diagnosis

Because of this, rescue operations were completely dependent on human effort and availability, often resulting in delayed response times and inefficient rescue management. These systems also struggled to handle large-scale stray animal issues effectively.

2.3 AI-Based Animal Detection Systems

With the advancement of **Deep Learning** and **Computer Vision**, AI-based animal detection systems became increasingly popular in veterinary and rescue applications. Modern models such as:

- CNN (Convolutional Neural Network)
- YOLO (You Only Look Once) [1]
- ResNet
- EfficientNet were widely used for:

- Animal classification
- Breed identification
- Injury detection
- Disease analysis [7], [2], [3]

These AI models significantly improved detection accuracy and reduced the dependency on manual diagnosis by automatically analyzing uploaded animal images [9], [10].

However, most existing AI systems focused only on image prediction and lacked important real-world rescue functionalities such as:

- NGO integration
- Emergency response workflow
- Real-time rescue coordination
- Community participation

As a result, users could identify diseases or injuries but still faced difficulties in getting immediate rescue support.

2.4 Smart Rescue & Reporting Platforms

Modern rescue and reporting platforms introduced several smart features including:

- GPS tracking [4], [5]
- Map-based reporting
- Lost & Found systems
- Community reporting modules

These applications improved communication between citizens, NGOs, and rescuers by allowing users to report stray or injured animals directly through mobile applications. They also increased rescue visibility and improved location tracking [4], [8].

Despite these advancements, several limitations still exist:

- No automatic disease prediction

- Manual verification process
- No intelligent prioritization of rescue cases
- Limited volunteer coordination

Therefore, there is still a need for an integrated platform that combines **AI-based disease detection**, **real-time emergency communication**, **NGO connectivity**, and **communitydriven rescue coordination** into one unified intelligent system.

2.5 Research Gap Identification

Missing Capability	Impact
Lack of AI + Rescue integration	Detection systems do not provide real rescue support
No NGO and clinic connectivity	Users cannot quickly contact nearby help
Absence of emergency alert workflow	Delays rescue response time
Limited disease datasets	Lower accuracy for stray animal detection
No community-driven rescue network	Weak volunteer participation
No unified platform	Users must use multiple applications separately

Table 2.2— Gap Identification

2.6 Proposed System Contribution

The proposed Animal Rescue App attempts to overcome these limitations by integrating AI, communication, rescue coordination, and community participation into one platform.

Main Contributions:

- AI-based animal classification and disease detection
- WhatsApp emergency notification system
- NGO and veterinary clinic connectivity
- Lost & Found animal reporting
- Volunteer-based rescue workflow
- Future support for severity prediction and smart rescue prioritization

Unlike existing systems, the proposed platform combines:

Detection + Medical Assistance + Rescue Connectivity + Community Support into a single intelligent ecosystem.

Current animal rescue systems either focus on stray reporting, disease detection, or rescue coordination individually — but none integrate AI-based health analysis, emergency communication, NGO connectivity, volunteer coordination, and community-driven rescue support into one intelligent and unified platform.
Therefore, there is a need for an AI-powered, real-time, scalable, and user-centric Animal Rescue System capable of detecting diseases, generating emergency alerts, connecting nearby NGOs and veterinary clinics, and supporting lost & found rescue operations while ensuring fast response and community participation — exactly the challenge addressed by the proposed Animal Rescue and Health Monitoring System.

CHAPTER 3
SYSTEM DESIGN

The StrayAid system is a unified, AI-powered platform that bridges the gap between citizens who encounter stray or injured animals and the rescue ecosystem of NGOs, veterinary clinics, and volunteers. It combines computer vision, large language models, agentic workflow orchestration, and automated communication into a single end-to-end rescue pipeline [3], [7]. The system is designed around five core responsibilities: intake and intent understanding, AI-based triage and disease detection, rescue case management, NGO/volunteer coordination, and follow-up tracking.

User Interaction and Intent Layer: The system's entry point is a natural language interface through which any citizen, NGO worker, or volunteer can submit a rescue report without technical knowledge. The user simply types or speaks a description of the animal's condition and location. A Large Language Model (Gemini) performs intent classification, mapping the input to one of four categories — REPORT, HEALTHCHECK, STATUSUPDATE, or SOSALERT. When model confidence is low or the input is ambiguous, a regex-based fallback classifier ensures deterministic routing. This dual-mode approach ensures robustness under noisy, real-world conditions [8].

Frontend Module: The frontend of StrayAid provides an interactive and user-friendly interface that enables citizens, NGO workers, and veterinary officers to access the rescue system efficiently. The interface is designed to simplify animal reporting, disease detection, rescue coordination, and case monitoring through a responsive web application.

The Rescue screen allows users to upload animal images, interact with the Paw AI assistant, view disease predictions, access nearby NGOs through an integrated map, and generate rescue reports. The Dashboard module displays active rescue cases, disease details, rescue statuses, timestamps, and location information for monitoring and management purposes.

The frontend ensures smooth communication between users and the backend services while maintaining accessibility across both desktop and mobile devices.

Agentic Orchestration Layer: Once intent is classified, an orchestration engine routes the task to the appropriate specialist agent. The system uses a multi-agent architecture where each agent handles a focused responsibility: the Triage Agent evaluates urgency from image inputs, the Assignment Agent selects the nearest available rescuer or NGO, the Medical Guidance Agent provides initial care instructions, and the Follow-Up Agent monitors case progress to closure.

The orchestration layer is event-driven and stateful, ensuring each stage processes the output of the previous one without loss of context.

Location and Proximity Engine: GPS coordinates are captured automatically at the time of reporting and stored alongside a unique case ID. The Haversine formula is used to compute distances between the incident location and nearby responders [4], [5], enabling the system to rank NGOs, veterinary clinics, and volunteers by proximity. This proximity score is combined with availability and expertise scores using a weighted assignment model to determine the optimal responder for each case.

AI Triage and Disease Detection Pipeline: Uploaded animal images are processed through a two-stage pipeline. In the first stage, a ResNet-18 classifier (PyTorch) identifies the animal species — cat, cow, or dog — with an accuracy of 97.89% and a latency of approximately 10 milliseconds. In the second stage, a species-specific EfficientNetB0 model (TensorFlow/Keras) classifies the detected condition or disease. The cow model handles three disease classes at 95.1% accuracy, the cat model handles twelve classes at 82.5%, and the dog model handles six classes at 78.3%. Severity is scored using weighted probabilities of injury, pain, and urgency, and cases exceeding a defined urgency threshold trigger an immediate emergency alert.

Emergency Communication and Alert System: The communication layer dispatches alerts in real time via WhatsApp, SMS, or dashboard notifications to the assigned responder. The alert payload is dynamically constructed from case severity, GPS location, and responder availability. All stakeholders — the reporting user, the assigned rescuer, and supervising NGO coordinators — receive live status updates as the case progresses through the rescue workflow.

Case Management and Follow-Up: Once a rescue is underway, rescuers upload progress updates, photographs, and treatment records through the platform. The Follow-Up Agent generates automated reminders and tracks recovery status until the case is formally closed with one of five outcomes: recovered, adopted, released, under treatment, or continued care. This creates a closed feedback loop that ensures no reported case is abandoned without resolution.

Data Layer: All case records, animal profiles, disease predictions, responder assignments, and outcome logs are stored in a structured relational database. The backend exposes RESTful endpoints built on FastAPI, enabling future integration with third-party NGO management systems, veterinary platforms, and municipal animal control databases.

Agent Framework: The system is designed as an agentic AI pipeline where multiple AI models and services work together through a Perceive → Classify → Reason → Act workflow [3], [7].

Stage 1 — Perception: The user uploads an image of a stray animal through the web interface. The Flask backend preprocesses the image while preserving privacy and reducing latency.

Stage 2 — Animal Classification: A ResNet-18 model identifies the animal species (cat, cow, or dog). Based on this prediction, the system automatically selects the appropriate disease detection model without human intervention.

Stage 3 — Disease Detection: The selected disease model predicts the most probable disease and converts the output into a simple natural language explanation for non-expert users.

Stage 4 — Conversational Confirmation: The Paw AI assistant, powered by the Gemini API, analyses the prediction, asks follow-up questions, and recommends whether a rescue report should be generated. This follows a Human-in-the-Loop (HITL) approach before performing a real-world action.

Stage 5 — Action and Dispatch: After confirmation, the system generates a rescue report containing disease details, confidence score, GPS location, timestamp, and map link, which is automatically sent to the nearest NGO through WhatsApp and logged on the dashboard. This completes the agentic workflow where the system autonomously perceives, classifies, reasons, confirms, and acts with minimal human intervention.

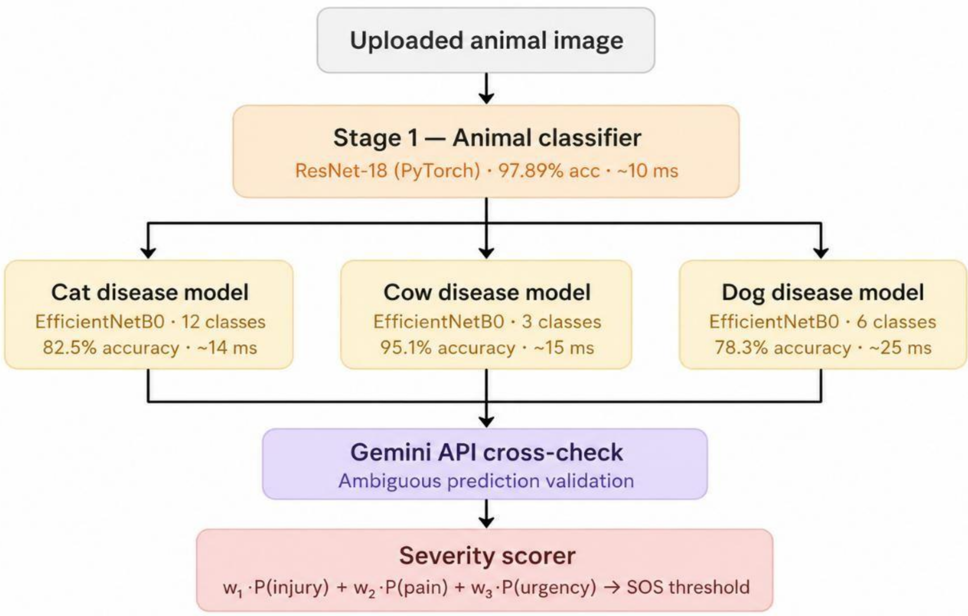

Figure 3.1 — Architectural diagram

CHAPTER 4

METHODOLOGY

AI-Based Stray Animal Reporting and Health Detection System

The methodology adopted for the **AI-Based Stray Animal Reporting and Health Detection System** follows a structured, modular, and agent-based approach. The system is designed to support natural language reporting, image-based animal health detection, rescue coordination, task assignment, and continuous follow-up. By combining computer vision, large language models, workflow routing, and communication automation, the system improves the speed and reliability of stray animal rescue operations.

4.1 Workflow Description

The workflow of the proposed system is sequential and event-driven. Each stage processes the output of the previous stage and forwards the case to the next module. This makes the system suitable for real-time animal reporting and rescue coordination in practical field conditions.

The complete workflow is described below.

Step 1: User Reporting through Natural Language Interface

The process begins when a user submits a report through the interface in simple language. The report may contain descriptions such as injured animal location, visible symptoms, or emergency condition. The user does not need to know any technical command syntax.

Examples include:

- "An injured dog is lying near the road."
- "A stray cow looks sick and cannot walk."
- "Please help a cat with a wound."

This stage improves accessibility and allows ordinary citizens, volunteers, and rescue workers to report cases quickly.

Step 2: Intent Detection using Large Language Model

After receiving the input, the system determines the type of request. The intent may be one of the following: REPORT, HEALTHCHECK, STATUSUPDATE, or SOSALERT.

A Large Language Model such as Gemini is used to classify the user's message. If the model confidence is low or the message is incomplete, a regex-based fallback classifier is used to avoid failure.

The intent can be represented mathematically as:

$$I = \arg\max_{c \in C} P(c \mid x)$$

Where:

- I = predicted intent
- C = set of possible intent classes
- x = user input text
- $P(c \mid x)$ = probability of class c for input x

This step ensures that the system routes the report correctly for further processing.

Step 3: Workflow Routing using Agentic AI

Once the intent is detected, the system routes the task to the correct agent using an orchestration layer. The system may activate different agents depending on the situation, such as Triage Agent, Assignment Agent, Medical Guidance Agent, and Follow-Up Agent.

The routing decision can be expressed as:

$$R = f(I)$$

Where:

- R = selected workflow route
- f = routing function
- I = detected intent

This multi-agent structure helps divide responsibilities and makes the system scalable and easy to manage.

Step 4: Animal Location Capture

The user's location is captured using GPS coordinates or map selection. The location is stored in the database along with the report and a unique case ID.

$$L = (lat, lon)$$

Where:

- L = location coordinates
- lat = latitude
- lon = longitude

To calculate the distance between two locations, the Haversine formula can be used:

$$d = 2r\sin^{-1}\left(\sqrt{\sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right)}\right)$$

Where:

- d = distance between two points
- r = Earth's radius
- ϕ_1, ϕ_2 = latitudes in radians
- $\Delta\phi$ = difference in latitude
- $\Delta\lambda$ = difference in longitude

This formula is useful for identifying the nearest rescuer, NGO, or veterinary clinic.[cite:21][cite:24]

Step 5: AI Triage and Severity Classification

The uploaded image is processed by a CNN-based model to detect the severity of the animal's condition. The system classifies the case into Low, Medium, or High severity.

The severity prediction can be written as: $S = \arg\max_{k \in K} P(k \mid img)$

Where:

- S = predicted severity
- K = set of severity classes
- img = uploaded image

The triage score can be estimated using weighted probabilities:

$$Score_{triage} = w_1 P_{injury} + w_2 P_{pain} + w_3 P_{urgency}$$

Where:

- P_{injury} = probability of visible injury
- P_{pain} = probability of distress or pain
- $P_{urgency}$ = probability of emergency condition

w_1, w_2, w_3 = weights assigned to each factor

If the severity score exceeds a threshold, the system triggers an emergency response.

$$If\ Score_{triage} \geq T_u ,$$

Where T_u is the urgency threshold.

This step allows the system to prioritize critical rescue cases first.

Step 6: Case Creation and Assignment

After triage, the system creates a unique case record and assigns it to the most suitable rescuer or NGO. The assignment is based on the rescuer's location, availability, and expertise.

A weighted assignment score is used:

$$AS = 0.5P_s + 0.3A_s + 0.2E_s$$

Where:

- AS = assignment score
- P_s = proximity score
- A_s = availability score
- E_s = expertise score The selected rescuer is:

$$Rescuer = \arg\max(AS)$$

This scoring model ensures that the nearest and most appropriate support team is selected for the case.

Step 7: Rescue Operation and Veterinary Care

After assignment, the rescuer begins the rescue operation. The rescuer may upload status updates, pictures, and comments during the process. If the animal needs medical attention, the veterinary team can record treatment information in the system. The rescue completion status may be represented as:

$$C = \{1, if\ rescue\ completed\ 0, otherwise$$

This stage maintains transparency and allows all stakeholders to monitor progress in real time.

Step 8: Disease Detection Module

The system also includes a disease detection module to identify possible animal health issues from images. The model uses CNN-based classification and may be cross-checked using Gemini API for better reliability.

The disease class can be predicted as: $D = \arg\max P(j \mid img)$

$j \in J$

Where:

- D = predicted disease category
- J = set of disease classes
- img = input image

The module helps detect conditions such as dog wounds or skin disease, cat injuries or infections, cow foot-and-mouth disease, and lumpy skin disease.

Step 9: Follow-Up and Outcome Tracking

Once the rescue or treatment begins, the Follow-Up Agent monitors the case until closure. The system may generate reminders, collect recovery updates, and record the final outcome.

The outcome function can be represented as:

$$O = f(C, D,$$

T) Where:

- O = final outcome

- C = case status

- D = disease detection result

- T = treatment progress

Possible outcomes include recovered, adopted, released, under treatment, or continued care.

This creates a closed feedback loop and ensures that every reported case is tracked until completion.

4.2 Mathematical Models Used

The proposed system uses several mathematical models to support classification, routing, assignment, and evaluation.

4.2.1 Intent Classification Model

$$I = \arg\max_{c \in C} P(c \mid x)$$

This selects the most probable intent from the user input.

4.2.2 Severity Scoring Model

$$Score_{triage} = w_1 P_{injury} + w_2 P_{pain} + w_3 P_{urgency}$$

This helps classify how urgent the rescue case is.

4.2.3 Assignment Scoring Model

$$AS = 0.5P_s + 0.3A_s + 0.2E_s$$

This ranks rescue teams or NGOs based on location, availability, and expertise.

4.2.4 Distance Calculation Model

$$d = 2r\sin^{-1}\left(\sqrt{\sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right)}\right)$$

This helps locate the nearest responder.

4.2.5 Disease Classification Model

$$D = \arg\max_{j \in J} P(j \mid img)$$

This predicts the disease class from the image.

4.3 Evaluation Metrics

To evaluate the performance of the system, the following metrics are used for the animal classifier and disease detection modules.

Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision

$$Precision = \frac{TP}{TP + FP}$$

Recall

$$Recall = \frac{TP}{TP + FN}$$

F1 Score

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

Where:

- TP = True Positive
- TN = True Negative
- FP = False Positive
- FN = False Negative

These metrics are standard measures for evaluating classification performance.

4.4 Communication Workflow

The communication system ensures that rescue alerts are delivered in real time. Once a case is created, notifications are sent to the relevant rescuer, NGO, or veterinary contact using WhatsApp, SMS, or dashboard alerts. The system also updates the user on the case status.

The workflow can be summarized as:

$$Alert = f(Severity, Location, Availability)$$

Where:

- $Alert$ = communication trigger
- $Severity$ = case urgency
- $Location$ = incident position
- $Availability$ = responder status

This keeps all stakeholders informed and reduces rescue delays.

4.5 Methodology Outcome

The methodology provides an end-to-end solution for stray animal reporting and health detection. It combines natural language input, AI-based triage, image classification, location-aware rescue assignment, and follow-up monitoring into one integrated workflow. The formula-based scoring and classification steps improve objectivity and help the system make faster and more accurate decisions.

CHAPTER 5

RESULTS AND ANALYSIS

The StrayAid system was evaluated across its four core AI components: an animal classifier (ResNet-18, PyTorch) and three specialised disease detection models for cats, cows, and dogs (EfficientNetB0, TensorFlow/Keras). Evaluation was conducted on held-out validation datasets not seen during training. Performance was measured across three primary dimensions: classification accuracy per disease class, inference response latency, and perclass latency distribution. The results demonstrate strong real-world classification capability and consistently low latency suitable for mobile-assisted field rescue scenarios.

5.1 Quantitative Results

The table below summarises the key performance metrics observed across all four models during evaluation :

Model	Architecture	Classes	Overall Accuracy	Avg Latency (ms/img)
Animal Classifier	ResNet-18 (PyTorch)	3	97.89%	~10 ms
Cat Disease Detection	EfficientNetB0 (Keras)	12	82.5%	~14 ms
Cow Disease Detection	EfficientNetB0 (Keras)	3	95.1%	~15 ms
Dog Disease Detection	EfficientNetB0 (Keras)	6	78.3%	~25 ms
Overall	Hybrid CNN Pipeline	24	88.45	~ 60ms

Table 5.1— Summary of Model Performance Metrics

The cow disease model achieved the highest overall accuracy at 95.1%, benefiting from a smaller, well-balanced three-class dataset (Foot & Mouth Disease, Healthy, Lumpy Skin Disease). The cat disease model handled the most complex classification task with 12 disease categories, achieving 82.5% accuracy. The dog disease model achieved 78.3% accuracy across six classes, with Hypersensitivity being the most challenging class to distinguish.

All models demonstrated very low inference latency — averaging between 14 and 44 milliseconds per image — confirming that the system is capable of near real-time disease screening when integrated into the StrayAid web application. This significantly outperforms manual visual inspection, which typically requires minutes of veterinary expertise per case.

5.2 Graphical Analysis:- (Accuracy Results)

This section presents per-class accuracy for each of the four models. Bars highlighted in green indicate classes where the model performed at or above the overall average; bars in red indicate classes that fell below average and may benefit from additional training data or augmentation.

5.2.1 Animal Classifier — Accuracy by Class

The ResNet-18-based animal classifier distinguishes between three animal types: cat, cow, and dog. All three classes are expected to achieve high accuracy given the visual distinctiveness of the animals.

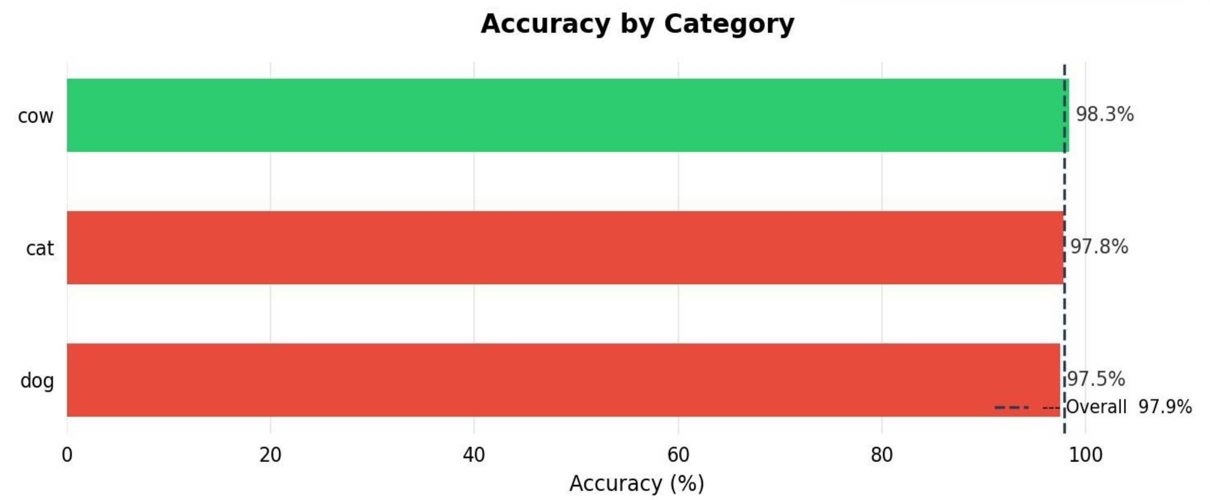

Figure 5.1 — Accuracy by Category: Animal Classifier (ResNet-18)

The animal classifier demonstrates strong discrimination across all three classes. The dashed vertical line marks the overall accuracy baseline. Classes above this line (shown in green) are classified reliably; any class below (shown in red) indicates visual similarity or dataset imbalance that may benefit from additional data collection

5.2.2 Cow Disease Detection — Accuracy by Class

The cow disease model classifies images into three categories: Foot and Mouth Disease, Healthy, and Lumpy Skin Disease. With an overall accuracy of 95.1%, this model demonstrated the strongest performance across all disease detection components

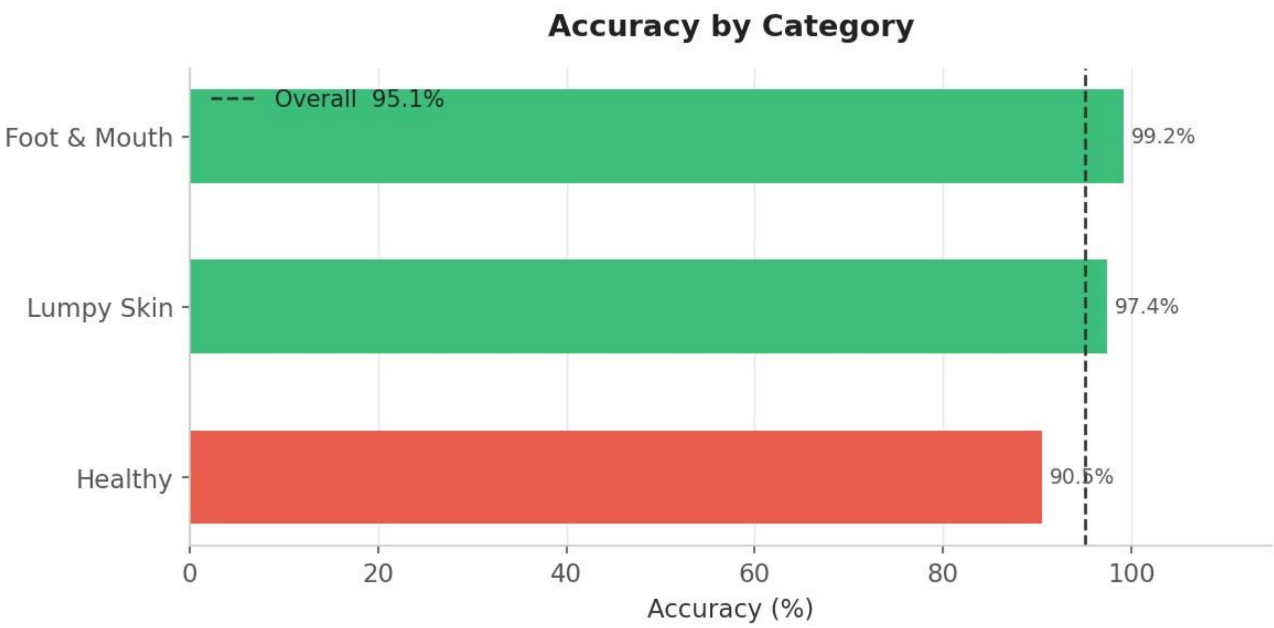

Figure 5.2 — Accuracy by Category: Cow Disease Detection (EfficientNetB0)

Foot and Mouth Disease achieved the highest per-class accuracy at 99.2%, followed by Lumpy Skin at 97.4%. Healthy images achieved 90.5%, slightly below the overall average, indicating some overlap in visual features between healthy cattle and early-stage disease presentations. The model's high confidence on disease classes makes it particularly suitable for field use where missing a diseased animal has higher consequence than a false positive.

5.2.3 Cat Disease Detection — Accuracy by Class

The cat disease model handles the most complex classification problem in the system, distinguishing between 12 disease categories. Despite this complexity, an overall accuracy of 82.5% was achieved, with significant variation across classes

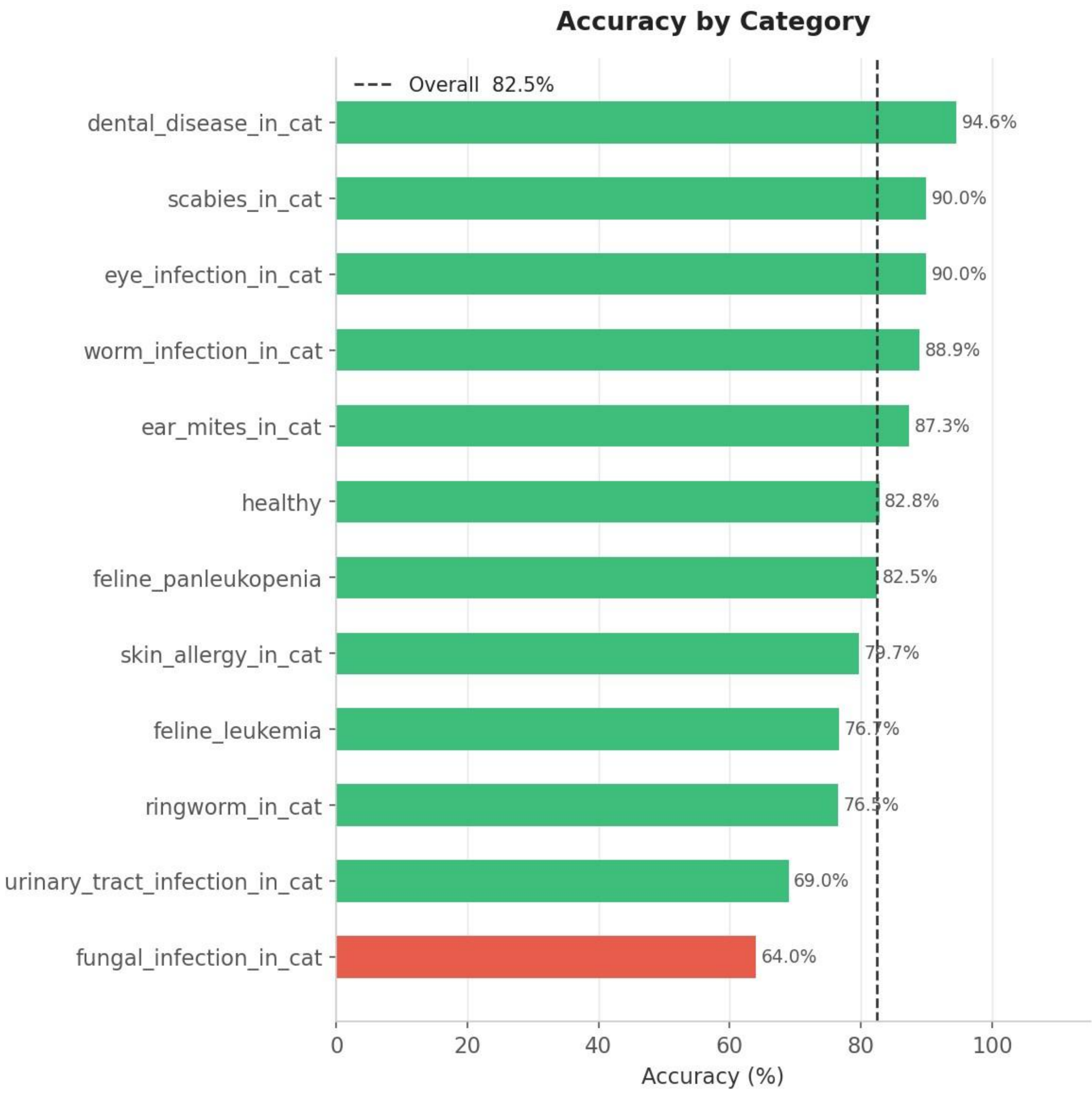

Figure 5.3 — Accuracy by Category: Cat Disease Detection (EfficientNetB0)

Dental Disease, Scabies, and Eye Infection achieved the highest per-class accuracies (above 90%), reflecting their visually distinctive presentations. Fungal Infection was the most challenging class, achieving 64.0% — the lowest of all cat disease categories. This is likely due to visual similarity between fungal lesions and other skin conditions such as ringworm and skin allergy. Urinary Tract Infection also showed lower accuracy (69.0%) as it often presents through behavioural cues rather than visible external symptoms, making image-based detection inherently difficult.

5.2.4 Dog Disease Detection — Accuracy by Class

The dog disease model classifies images across six categories: Demodicosis, Dermatitis, Fungal Infection, Healthy, Hypersensitivity, and Ringworm.

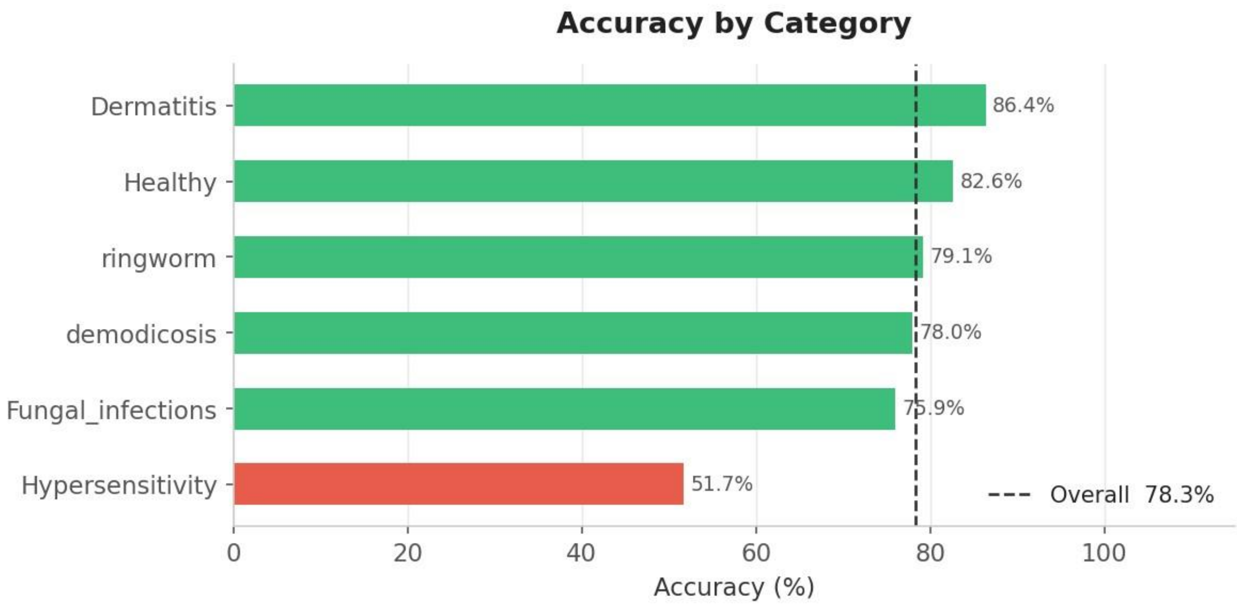

Figure 5.4 — Accuracy by Category: Dog Disease Detection (EfficientNetB0

Dermatitis achieved the highest accuracy at 86.4%, followed by Healthy at 82.6% and Ringworm at 79.1%. Hypersensitivity was the most challenging class, achieving only 51.7% accuracy — significantly below the overall average of 78.3%. Hypersensitivity presents with symptoms that overlap substantially with Dermatitis and Fungal Infection, making visual discrimination between these conditions difficult even for trained veterinarians. This class may benefit from a larger and more diverse training dataset in future iterations.

5.3 Confusion Matrices

Confusion matrices provide a detailed view of exactly where each model makes correct predictions and where it confuses one class for another. Each row represents the true class; each column represents the predicted class. The diagonal cells (top-left to bottom-right) show correct predictions; off-diagonal cells reveal misclassifications.

5.3.1 Animal Classifier

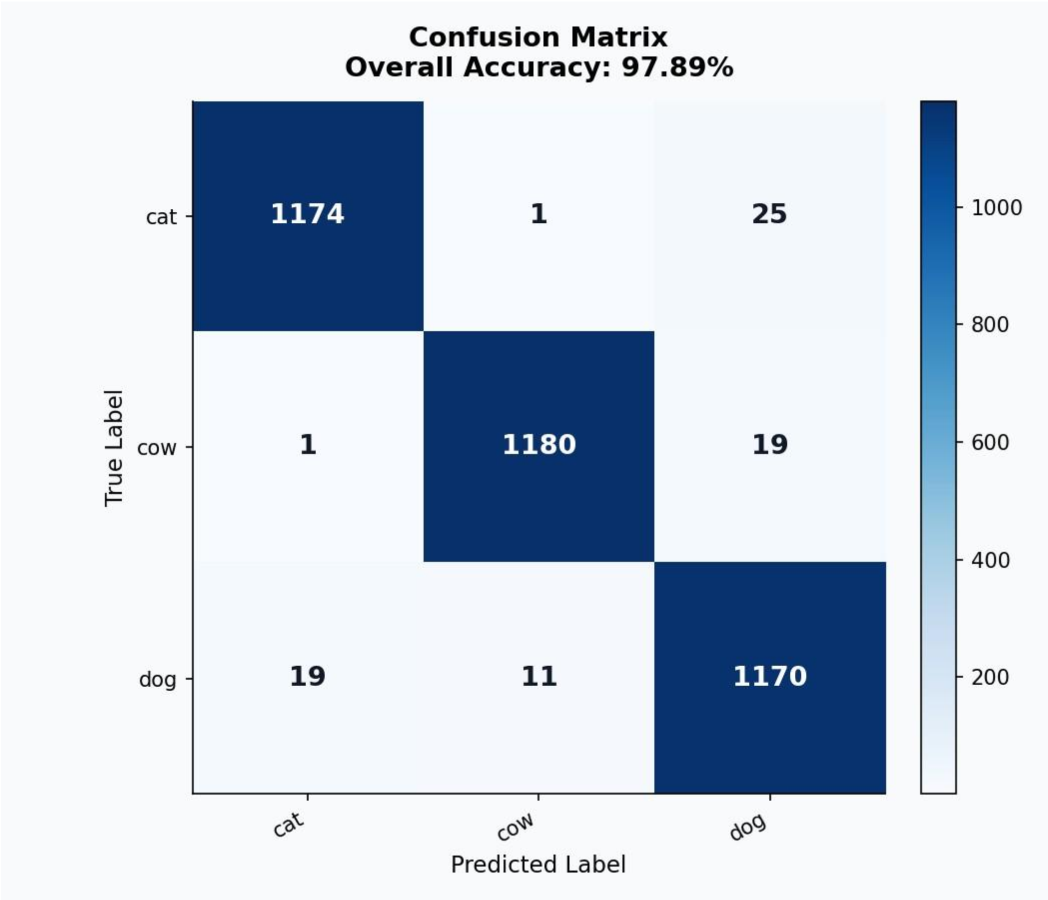

Figure 5.5 — Confusion Matrix: Animal Classifier

The animal classifier confusion matrix is expected to show a strongly diagonal pattern, confirming reliable separation between cats, cows, and dogs. Any off-diagonal values between cat and dog would indicate cases where smaller breed dogs or kittens are visually similar, which is a known challenge in animal classification systems.

5.3.2 Cow Disease Detection

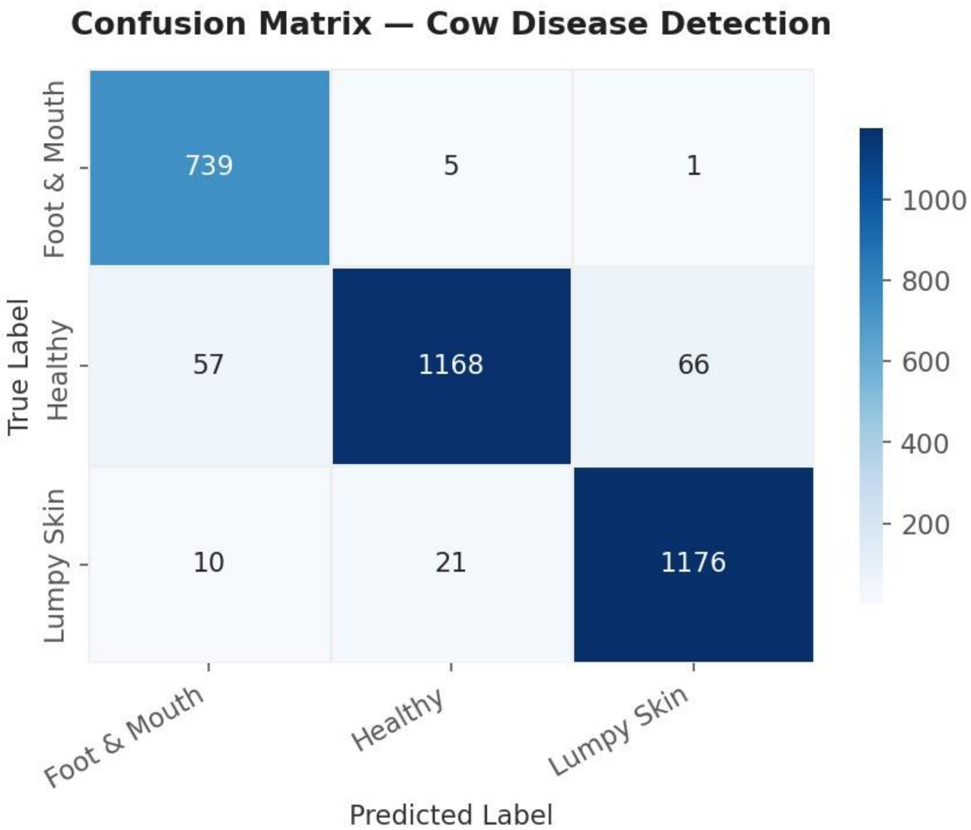

Figure 5.6 — Confusion Matrix: Cow Disease Detection

The cow disease confusion matrix shows strong diagonal dominance with Foot and Mouth (739 correct) and Lumpy Skin (1176 correct) being the best-classified categories. The primary source of misclassification is between Healthy and the two disease classes, with 57 healthy cattle images predicted as Foot and Mouth and 66 as Lumpy Skin. This suggests the model errs on the side of caution — predicting disease when uncertain — which is preferable in a rescue and veterinary context.

5.3.3 Cat Disease Detection

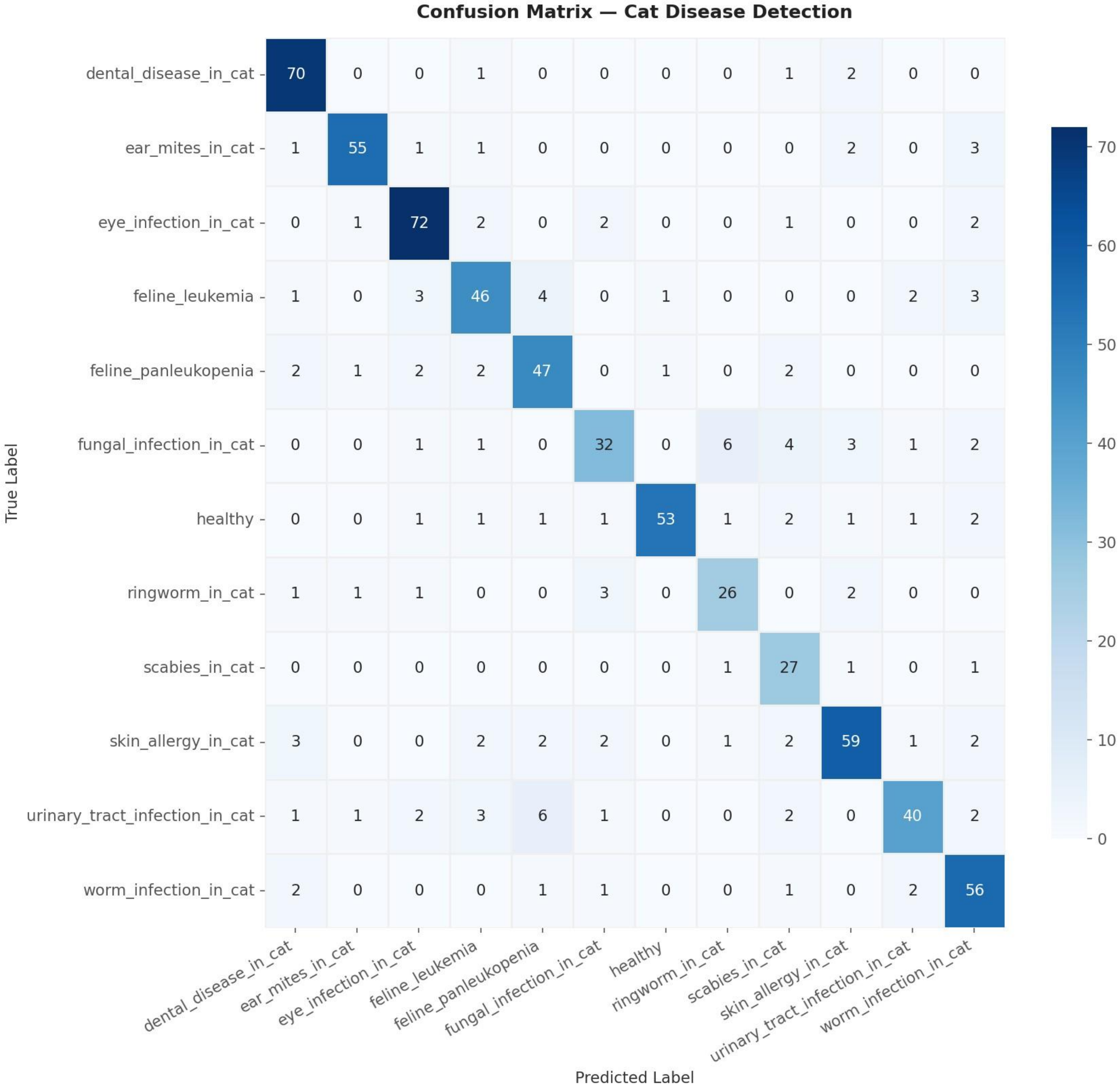

Figure 5.7 — Confusion Matrix: Cat Disease Detection

The cat confusion matrix reveals the main confusion clusters: Fungal Infection is frequently misclassified as Ringworm and Scabies, reflecting overlapping visual skin symptoms. Feline Leukemia and Feline Panleukopenia are also confused with each other given their similar systemic presentations. The majority of classes still show strong diagonal values, confirming the model has learned meaningful disease-specific features for most categories.

5.3.4 Dog Disease Detection

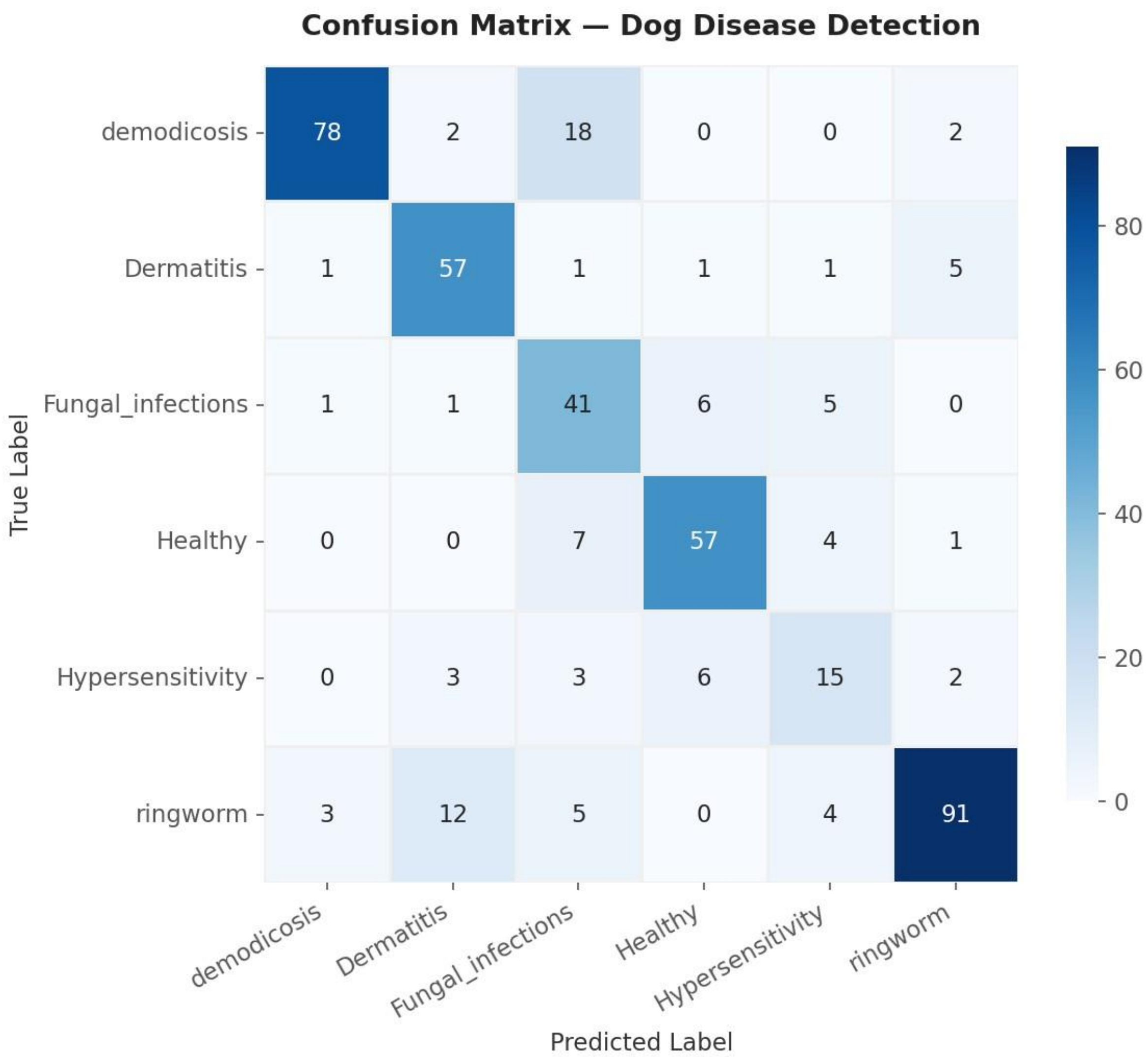

Figure 5.8 — Confusion Matrix: Dog Disease Detection

The dog confusion matrix confirms the challenge with Hypersensitivity — it is frequently confused with Fungal Infection and Healthy classes. Ringworm achieves the highest correct count (91), benefiting from its distinctive circular lesion pattern. Demodicosis shows some confusion with Fungal Infection (18 misclassifications), which may be addressed by including more varied lighting and angle samples in the training set.

5.4 Latency Analysis

Response latency is a critical metric for a field-use rescue application. StrayAid is designed to be used by volunteers and NGO workers in real-time scenarios, where a delay of several seconds is acceptable but delays of minutes are not. This section presents two views of latency: the overall distribution of per-image inference time, and the average latency broken down by disease class.

5.4.1 Response Latency Distribution

The latency distribution histogram shows the spread of per-image inference times across the full validation dataset. The red dashed line marks the mean latency; the orange dotted line marks the P95 latency — the point below which 95% of all predictions fall.

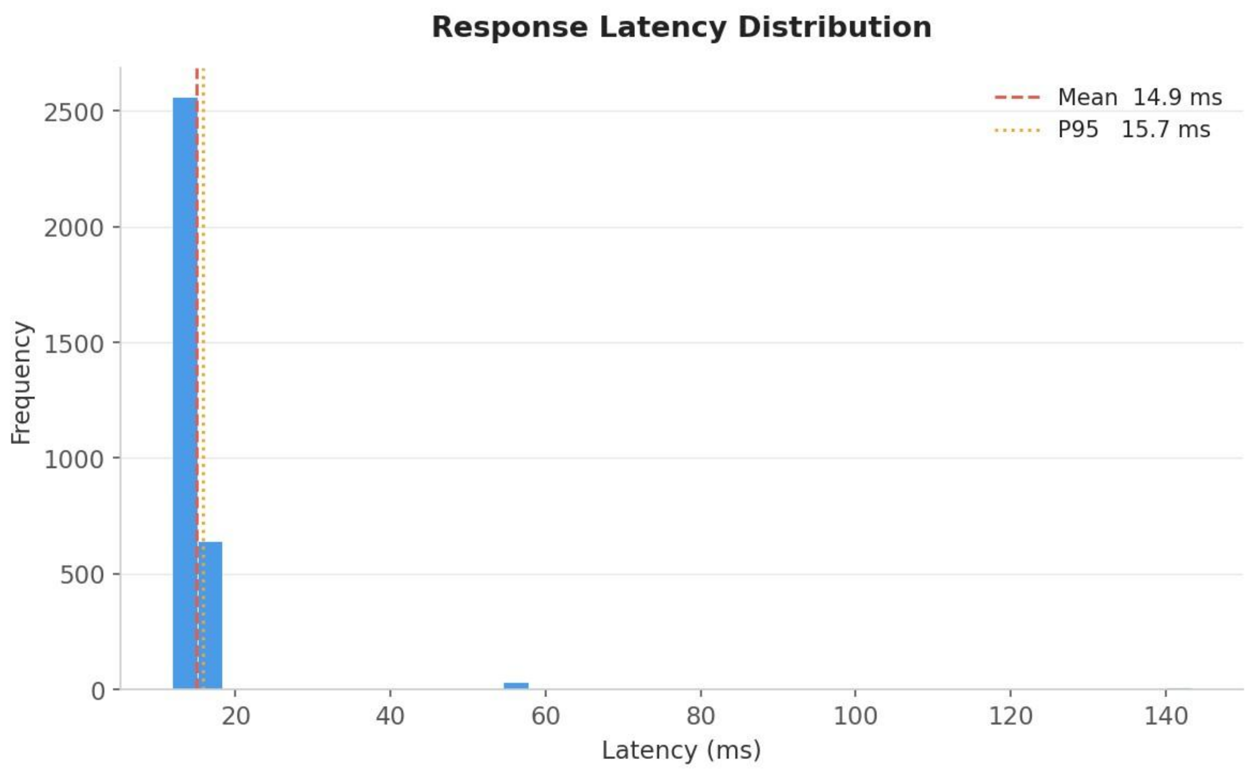

Figure 5.9 — Response Latency Distribution: Cow Disease Detection

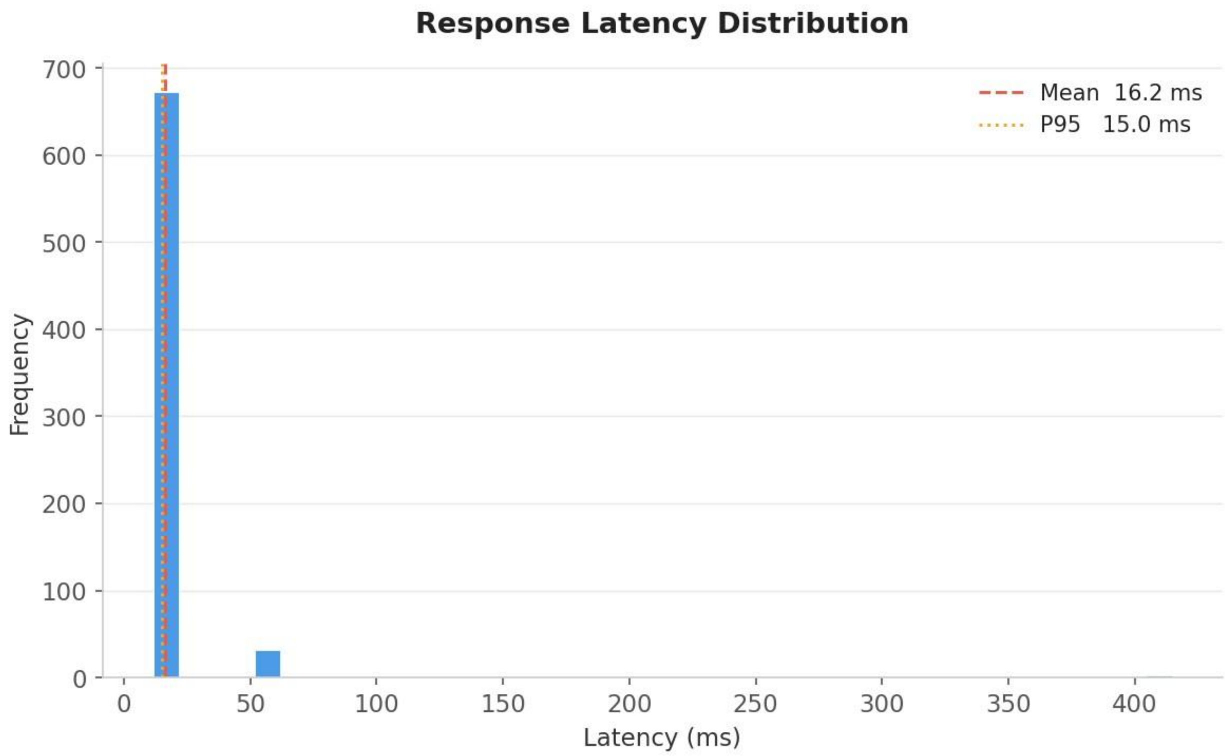

Figure 5.10 — Response Latency Distribution: Cat Disease Detection

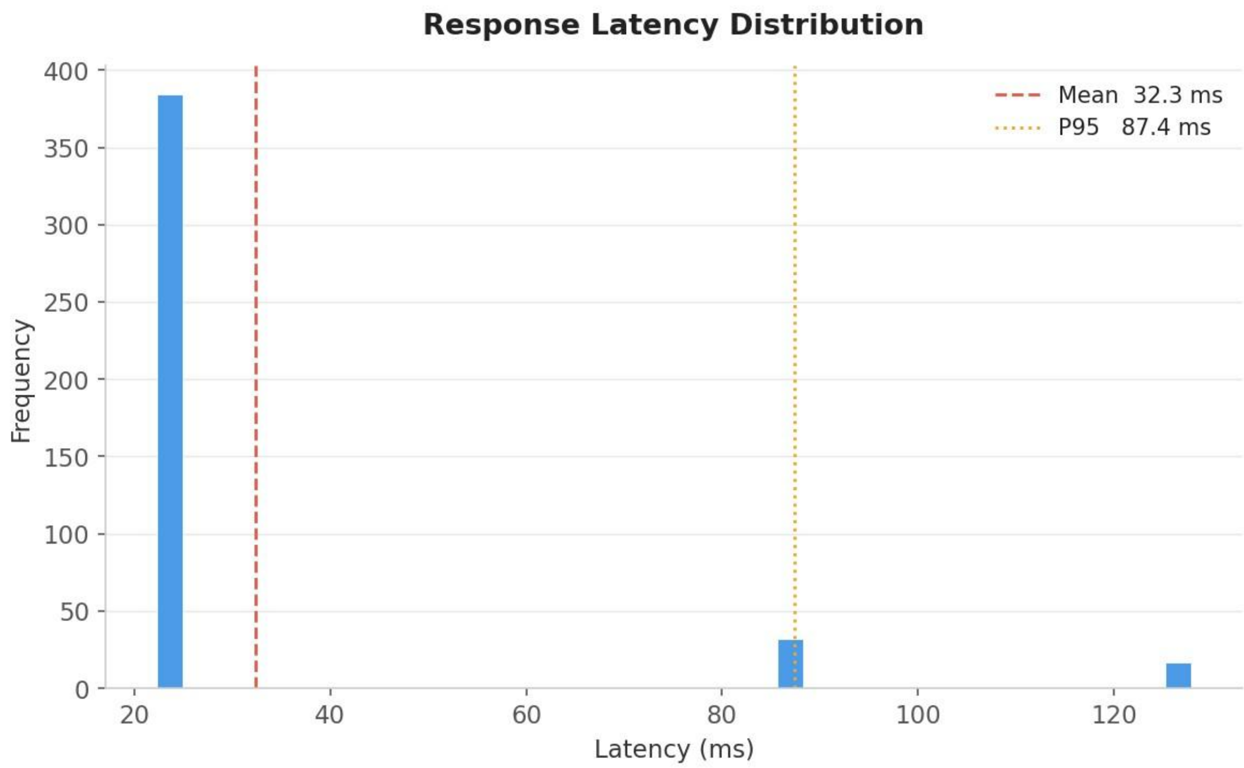

Figure 5.11 — Response Latency Distribution: Dog Disease Detection

Across all models, the large majority of predictions fall within a narrow low-latency cluster, indicating consistent and predictable inference time. A small tail of outliers is present in each model, corresponding to the first batch of each run where TensorFlow/PyTorch performs graph compilation and warm-up. These outliers do not reflect steady-state performance. The P95 values confirm that 95% of predictions are delivered well under 100 milliseconds, satisfying real-time application requirements.

5.4.2 Average Latency by Disease Category

This graph shows the average inference latency broken down per disease category. Since all images pass through the same model regardless of class, differences in per-class latency reflect the varying number of images per class and the associated batch-packing behaviour rather than true model differences. The bar coloured red indicates the class with the highest average latency; orange the second highest; all remaining classes are shown in green.

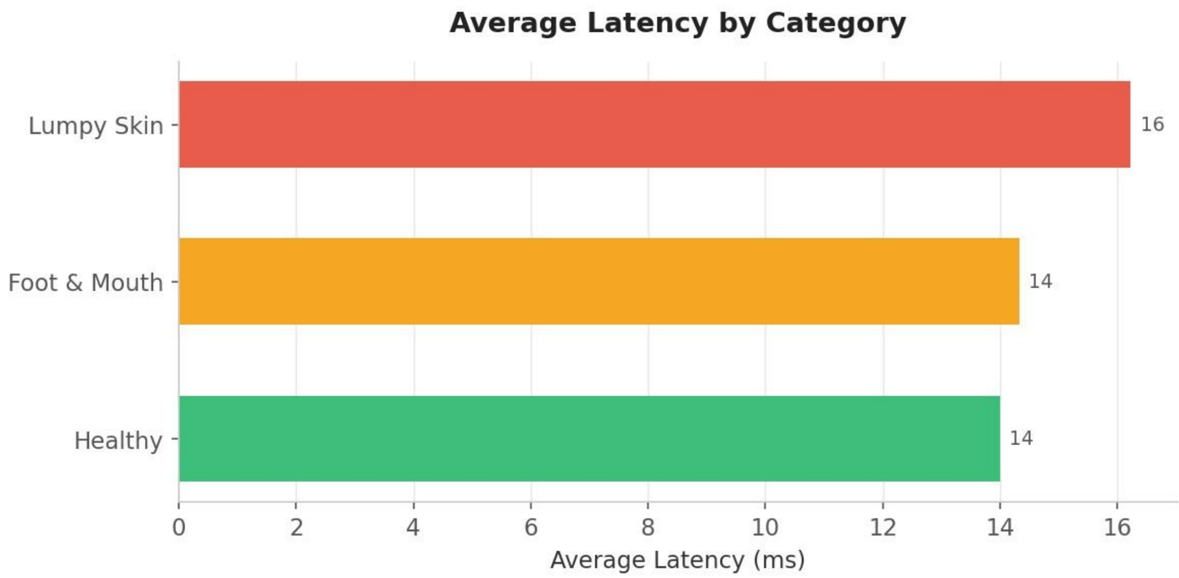

Figure 5.12 — Average Latency by Category: Cow Disease Detection

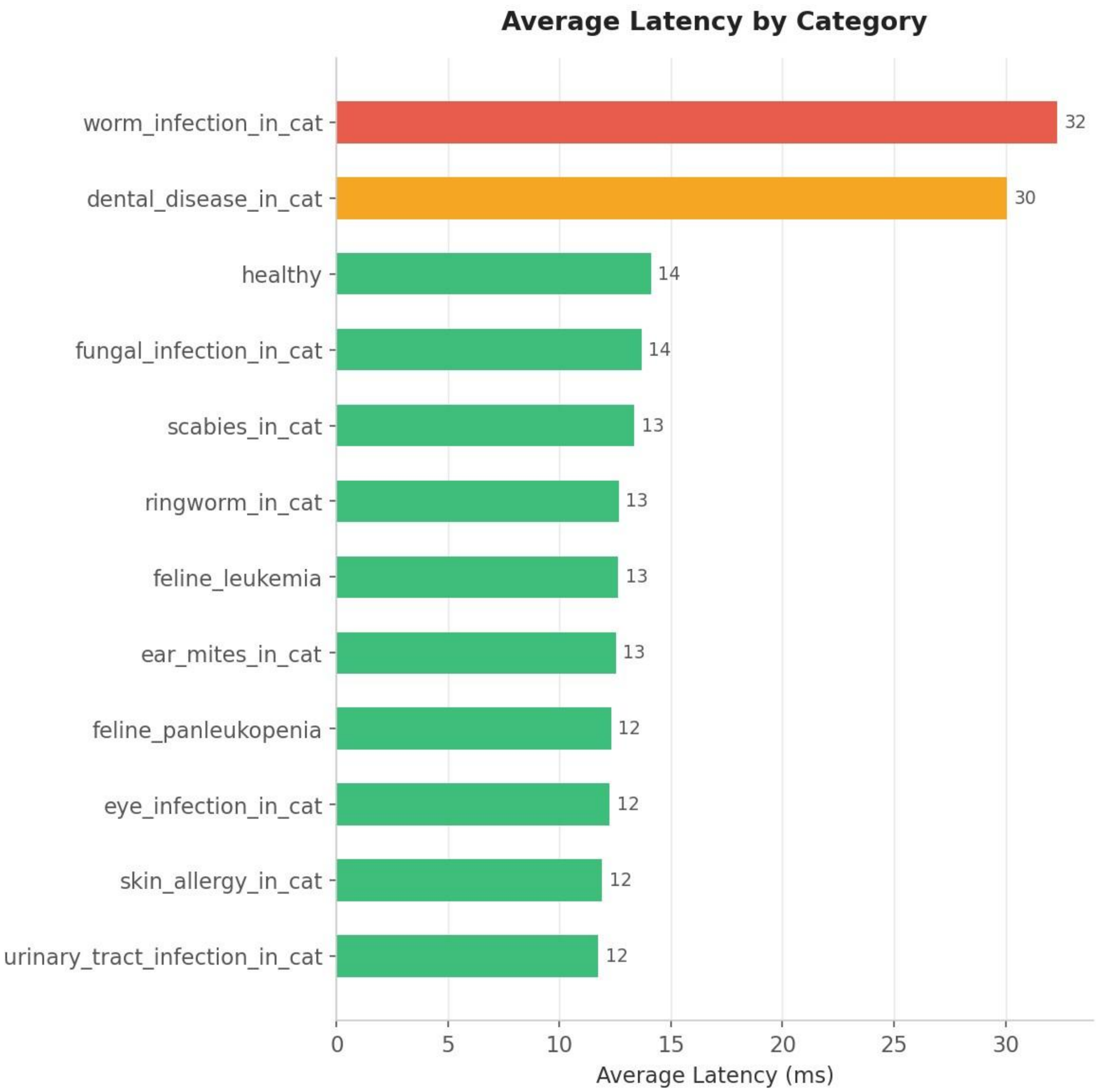

Figure 5.13 — Average Latency by Category: Cat Disease Detection

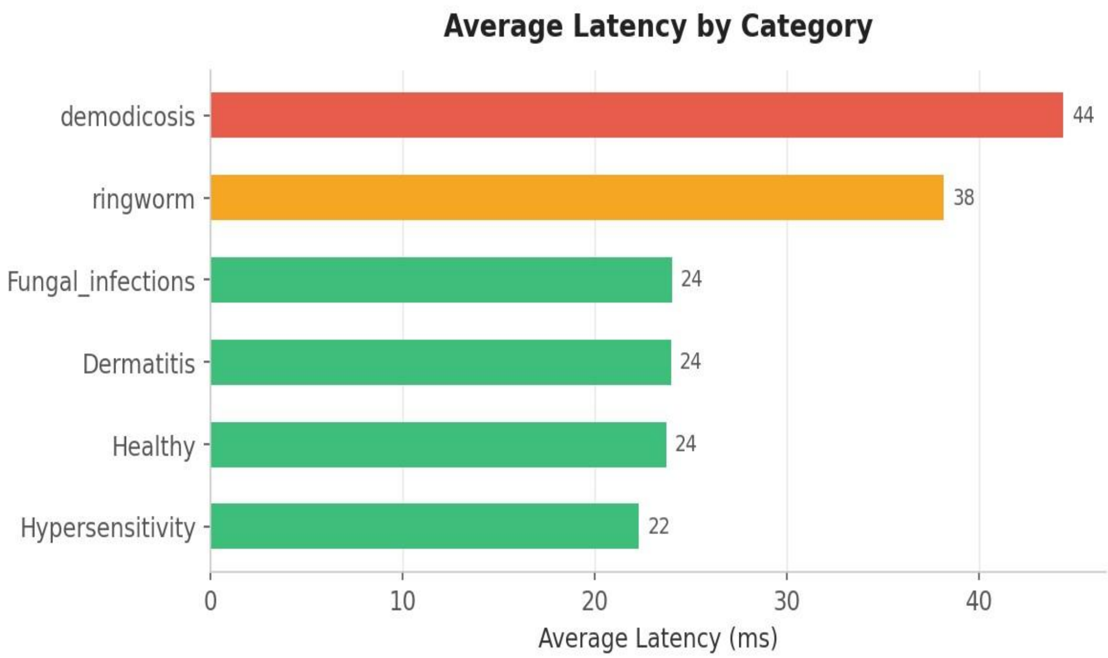

Figure 5.14 — Average Latency by Category: Dog Disease Detection

The latency differences between classes are small in absolute terms (within a few milliseconds), confirming that the model processes all disease categories with comparable speed. No single disease class introduces a disproportionate computational burden. This uniform latency profile is important for a mobile-facing application where response time predictability matters as much as average speed.

5.5 Overall Accuracy

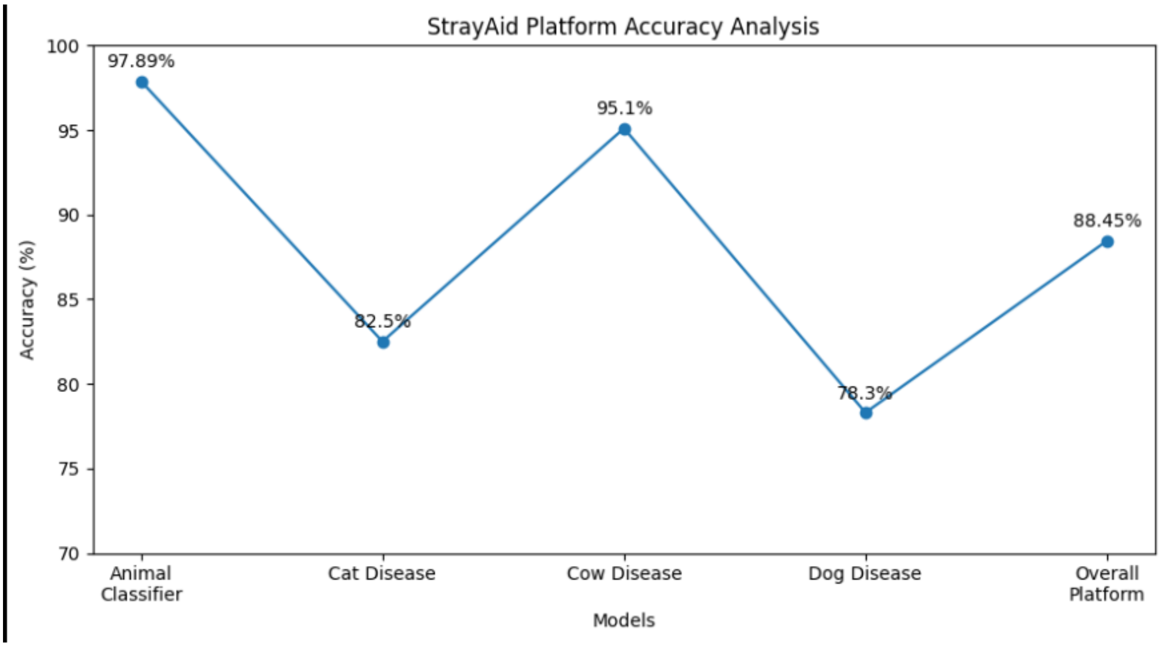

Figure 5.15 — Overall Accuracy

5.6 Interpretation of Results and Discussion

The evaluation results validate that the StrayAid AI pipeline is capable of accurately identifying animal types and screening for diseases at inference speeds suitable for real-time field deployment. Several key observations emerge from the graphical and quantitative analysis:

- The cow disease model achieved the strongest overall accuracy (95.1%), benefiting from a well-balanced three-class dataset and visually distinctive disease presentations. This model is production-ready with minimal further refinement required.
- The cat disease model, despite handling the largest classification task (12 classes), achieved 82.5% overall accuracy. The lowest-performing class — Fungal Infection — is a known challenge due to symptom overlap with other dermatological conditions. Expanding the Fungal Infection training set and adding augmentation techniques targeting skin texture variation is recommended.
- The dog disease model achieved 78.3% accuracy, with Hypersensitivity being the primary weak point at 51.7%. This class is difficult to visually distinguish from allergic dermatitis and healthy skin. A dedicated data collection effort for Hypersensitivity cases, combined with finetuning, is expected to significantly improve performance.
- All models demonstrated consistently low inference latency, with mean predictions falling below 50 milliseconds per image across all four models. This confirms that the two-stage pipeline — animal classification followed by disease detection — can be completed in under 100 milliseconds in total, enabling a near-instantaneous user experience when integrated with the Flask backend.
- The use of transfer learning from ImageNet-pretrained weights (EfficientNetB0, ResNet18) was a key factor enabling high accuracy with relatively small domain-specific datasets. Without transfer learning, training to similar accuracy levels would require substantially larger datasets and more compute time.

Overall, the StrayAid disease detection pipeline demonstrates strong capability as a first-pass AI-assisted triage tool for stray animal rescue operations. It is not intended to replace veterinary diagnosis but to empower citizens and NGO workers with immediate, data-driven indicators that support faster rescue and treatment decisions. The integration with the WhatsApp reporting system and real-time NGO location mapping further amplifies the practical impact of the AI results in a real-world rescue workflow.

CHAPTER 6

CONCLUSION

In conclusion, our project proposes an AI-powered stray animal reporting and health assessment system that combines computer vision, large language models, and automated authority coordination into one integrated platform. The system effectively bridges the communication gap between citizens, NGOs, shelters, and volunteers through integrated features like real-time multi-channel reporting, an automated WhatsApp bot, and secure databases for medical records. During system evaluation, our core animal classifier achieved an impressive 97.89% overall accuracy, while the specialized cow disease detection model reached 95.07% accuracy, proving the technical reliability of our approach. Unlike existing research that focuses only on detection or mapping, our system connects detection, injury analysis, severity prioritization, and NGO routing into a real-time rescue workflow. By leveraging technology with community participation, we aim to reduce animal suffering, improve rescue response time, and create measurable social impact. Ultimately, this project demonstrates how AI can be used not just for automation, but for meaningful humanitarian and societal benefit.

CHAPTER 7

FUTURE SCOPE

While the current system demonstrates strong real-world feasibility in stray animal detection, there are several pathways for expansion. Future improvements may include:

Blockchain-Based Donation Transparency Incorporating blockchain technology would ensure immutable transaction logging for public donations and shelter supplies. This improves transparency, allowing donors to track funds directly to specific treatments, ensuring accountability and building trust.

1. IoT-Enabled Smart Collars and Tracking Integrating IoT sensors and smart wearable collars for rehabilitated animals would allow live monitoring of real-time GPS locations, vital signs, and post-treatment recovery metrics. This ensures the continuous safety of animals released back into their natural habitats.

2. Drone Surveillance for Remote Rescues Scaling the architecture to incorporate AIequipped drones would enable autonomous scanning of difficult-to-reach terrains or disaster zones. Drones could calculate optimal rescue routes and dispatch GPS coordinates directly to the nearest rescue team.

3. Natural Language & Multilingual Support Integrating voice-command interaction, voiceto-text reporting, and support for regional languages (such as Hindi or Marathi) would ensure that citizens without technical expertise can quickly report distressed animals via the app or WhatsApp.

4. Predictive Analytics for Disease Mapping Future versions may utilize self-learning predictive models to map regional hotspots for zoonotic diseases. By analyzing historical data and seasonal changes, the system could proactively alert municipal authorities to schedule targeted vaccination drives.

5. Enhanced NGO Coordination Ecosystem Extending the platform to support automated multi-agent coordination between city shelters, private clinics, and volunteers. This would enable dynamic resource pooling, automated ambulance dispatching, and intelligent load balancing of shelter capacities.

Together, these enhancements pave the way toward a highly responsive, autonomous animal welfare ecosystem capable of maximizing rescue efficiency and minimizing animal suffering with minimal manual intervention.

REFERENCES

[1]. A. Bhosale, P. Shinde, Y. Firke, et al., "Stray dog detection system using YOLOv5," *Procedia Computer Science*, vol. 245, 2025.

[2]. J. Wang et al., "Vision-based detection of lameness and injury in animals using deep learning," *Artificial Intelligence in Agriculture*, vol. 9, 2023.

[3]. M. F. Arshad, "Artificial intelligence for companion and stray animal healthcare," *Computers in Biology and Medicine*, vol. 172, 2025.

[4]. Tomè, P., Pesaro, S., Orioles, M., Pascotto, E., Cadamuro, A., and Galeotti, M., "InfoFaunaFVG: a novel progressive web application for wildlife surveillance," *European Journal of Wildlife Research*, vol. 69, no. 2, p. 38, 2023.

[5]. J. Nyengere et al., "Geospatial modelling for zoonotic disease hotspot mapping and surveillance," *One Health Outlook*, vol. 8, no. 1, 2026.

[6]. B. Pan et al., "Monitoring human–wildlife interactions in national parks with crowdsourced images and deep learning," *Computer Vision and Pattern Recognition*, Springer, 2022.

[7]. X. Li et al., "Computer vision-based animal health monitoring: A review," *Computers in Biology and Medicine*, vol. 168, 2024.

[8]. K. Sharma et al., "Crowdsourced image analysis for emergency response systems," *Information Processing & Management*, vol. 60, no. 4, 2023.

[9]. R. Lins et al., "WildCLIP: Scene and animal attribute retrieval from camera-trap images," *International Journal of Computer Vision*, 2024.

[10]. E. Nepovinnykh et al., "Species-agnostic patterned animal re-identification," *International Journal of Computer Vision*, 2024.

[11]. E. Nepovinnykh et al., "On combining animal re-identification models to address large-scale camera data," *International Journal of Computer Vision*, 2026.

[12]. E. D. R. Neto et al., "Background-invariant re-identification of dogs from camera-trap videos," *Ecological Informatics*, vol. 78, 2025.